

Git & GitHub Commands - Quick Guide

One-line explanation + practical example for each command

Typical workflow: create folder -> git init -> add files -> git add -> git commit -> connect GitHub -> git push

1. Git Setup and Repository Creation

Command	One-line explanation	Example
<code>git --version</code>	Checks whether Git is installed and shows its version.	<code>git --version</code>
<code>git config --global user.name "Your Name"</code>	Sets the name that will appear in your Git commits.	<code>git config --global user.name "Renuka"</code>
<code>git config --global user.email "you@example.com"</code>	Sets the email associated with your Git commits.	<code>git config --global user.email "renuka@example.com"</code>
<code>git config --list</code>	Displays your current Git configuration settings.	<code>git config --list</code>
<code>mkdir project-name</code>	Creates a new folder for your project.	<code>mkdir stories</code>
<code>cd project-name</code>	Moves into the project folder.	<code>cd stories</code>
<code>git init</code>	Initializes the current folder as a local Git repository.	<code>git init</code>
<code>git status</code>	Shows changed, staged, and untracked files in the repository.	<code>git status</code>
<code>git add filename</code>	Stages one file so it can be included in the next commit.	<code>git add story.html</code>
<code>git add .</code>	Stages all changed and new files in the current folder.	<code>git add .</code>
<code>git commit -m "message"</code>	Saves the staged changes as a version in Git history.	<code>git commit -m "Add first HTML page"</code>
<code>git log</code>	Shows the commit history of the repository.	<code>git log</code>

2. Viewing Changes and Commit History

Command	One-line explanation	Example
<code>git log --oneline</code>	Shows the commit history in a short one-line format.	<code>git log --oneline</code>
<code>git diff</code>	Shows changes made in files that are not yet staged.	<code>git diff</code>
<code>git diff --staged</code>	Shows changes that are staged and ready to commit.	<code>git diff --staged</code>
<code>git branch</code>	Lists all local branches and highlights the current branch.	<code>git branch</code>

3. Branching and Merging

Command	One-line explanation	Example
<code>git branch branch-name</code>	Creates a new branch without switching to it.	<code>git branch feature-home</code>

Command	One-line explanation	Example
git switch branch-name	Switches from the current branch to another branch.	git switch feature-home
git switch -c branch-name	Creates a new branch and immediately switches to it.	git switch -c feature-login
git checkout branch-name	Older command used to switch to an existing branch.	git checkout main
git checkout -b branch-name	Older command used to create and switch to a new branch.	git checkout -b feature-about
git merge branch-name	Merges another branch into the branch you are currently on.	git merge feature-home
git branch -d branch-name	Deletes a local branch after it has been merged.	git branch -d feature-home

4. Connecting to GitHub and Synchronizing

Command	One-line explanation	Example
git remote -v	Displays the remote repositories linked to your local project.	git remote -v
git remote add origin URL	Connects your local repository to a GitHub repository.	git remote add origin https://github.com/username/stories.git
git branch -M main	Renames the current branch to main.	git branch -M main
git push -u origin main	Pushes the main branch to GitHub and sets the upstream branch.	git push -u origin main
git push	Uploads new local commits to the already connected GitHub branch.	git push
git pull	Downloads remote changes and merges them into your current branch.	git pull
git fetch	Downloads remote updates without automatically merging them.	git fetch
git clone URL	Copies an existing GitHub repository to your computer.	git clone https://github.com/username/stories.git

5. File Management and Undo Commands

Command	One-line explanation	Example
git rm filename	Deletes a file and stages the deletion for the next commit.	git rm old.html
git mv old new	Renames or moves a file and stages the change.	git mv home.html index.html
git restore filename	Discards unstaged changes in a working file.	git restore story.html
git restore --staged filename	Removes a file from the staging area without deleting its changes.	git restore --staged story.html
git reset --soft HEAD~1	Removes the latest commit but keeps its changes staged.	git reset --soft HEAD~1

6. Tags / Release Marking

Command	One-line explanation	Example
git tag tag-name	Creates a tag, often used to mark a release version.	git tag v1.0
git push origin tag-name	Uploads a local tag to GitHub.	git push origin v1.0

Complete Example: Create a Project and Push it to GitHub

1. `mkdir stories`
2. `cd stories`

```
3.git init
4.echo "<html><body><h1>My Story</h1></body></html>" > story.html
5.git add story.html
6.git commit -m "Add story page"
7.git branch -M main
8.git remote add origin https://github.com/username/stories.git
9.git push -u origin main
```

After the first push: make changes -> git add . -> git commit -m "message" -> git push.

Useful Notes

- Run Git commands inside the project folder unless a command is explicitly global.
- Create the GitHub repository first if you want to use git remote add origin with its URL.
- Use meaningful commit messages such as "Fix login validation" instead of "changes".
- Before merging, switch to the branch that should receive the changes; for example, switch to main and then merge feature-home.
- Use git status frequently - it is the easiest way to understand what Git is currently tracking.